

PRAKTIKUM SISTEM OPERASI

MODUL 4

MEMBACA SOURCE CODE XINU



Oleh:

Zhafir Zaidan Avail

2311104059

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

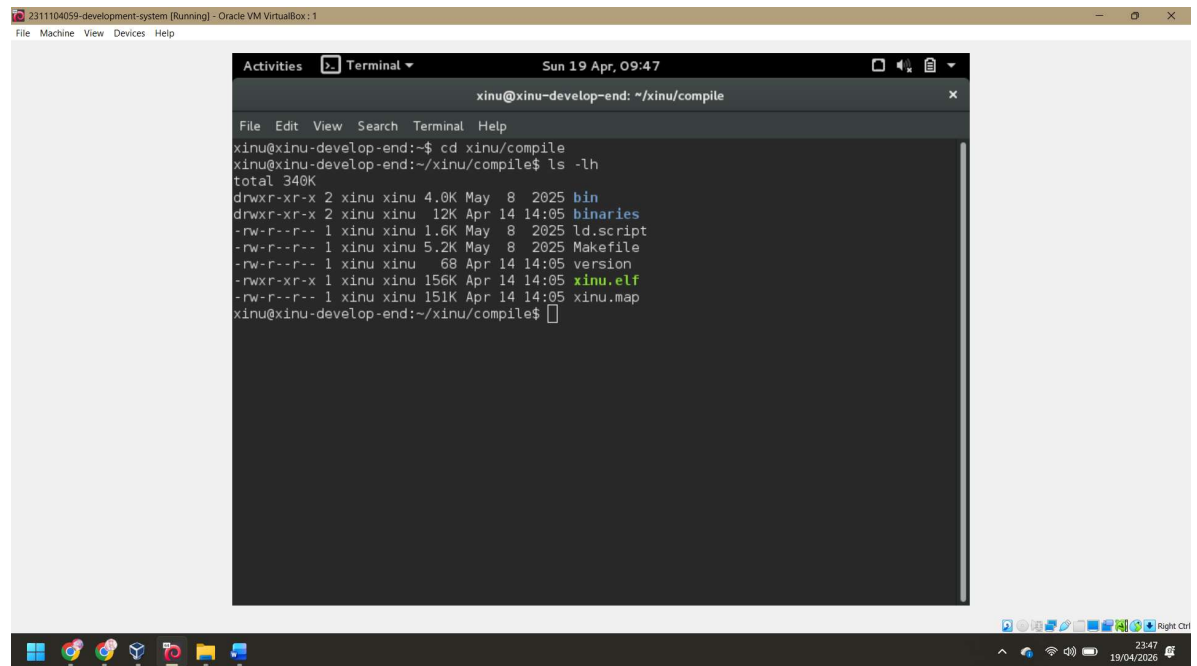
JURNAL/TP

1. Soal 1

Apa nama image yang dihasilkan setelah melakukan kompilasi pada Xinu?

Berapa ukuran file tersebut? Ada pada folder apa file image tersebut?

Hint: baca kembali modul-modul sebelumnya



The screenshot shows a terminal window titled "xinu@xinu-develop-end: ~/xinu/compile" with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal output shows the result of the command `ls -lh` in the `~/xinu/compile` directory. The files listed are:

Permissions	Number of Files	Owner	Group	Size	Month	Day	Time	File Name
drwxr-xr-x	2	xinu	xinu	4.0K	May	8	2025	bin
drwxr-xr-x	2	xinu	xinu	12K	Apr	14	14:05	binaries
-rw-r--r--	1	xinu	xinu	1.6K	May	8	2025	ld.script
-rw-r--r--	1	xinu	xinu	5.2K	May	8	2025	Makefile
-rw-r--r--	1	xinu	xinu	68	Apr	14	14:05	version
-rwxr-xr-x	1	xinu	xinu	156K	Apr	14	14:05	xinu.elf
-rw-r--r--	1	xinu	xinu	151K	Apr	14	14:05	xinu.map

The terminal prompt is `xinu@xinu-develop-end:~/xinu/compile$`. The window title bar indicates the system is "2311104059-development-system [Running]" and the date is "Sun 19 Apr, 09:47". The bottom of the screen shows a taskbar with various application icons and a system tray with the date "19/04/2026" and time "23:47".

2. Membaca source code Xinu

Hasil overview:

The screenshot displays the Sourcetrail application interface for the Xinu project. The central pane shows a hierarchical overview of the project's components, including Files (232), Macros (179), Structs (5), Unions (1), Functions (404), Global Variables (15), Typedefs (1), and Enums (1). The right pane provides project statistics: 232 files (20 complete), 22502 lines of code, 1386 symbols, 2831 references, and 282 errors (202 fatal). A warning message indicates that the project has fatal errors, which cause a lot of missing information in affected files. The bottom pane shows a list of errors, including a fatal error for the 'xinu.h' file not found and a device/ethcontrol.c error. The status bar at the bottom indicates 'displaying 202 errors (202 fatal)' and 'No IDE connected'.

Berdasarkan hasil eksplorasi menggunakan Sourcetrail, project Xinu terdiri dari ratusan file dan fungsi. Struktur kode terbagi menjadi beberapa kategori seperti file, fungsi, variabel global, dan macro. Hal ini menunjukkan bahwa Xinu merupakan sistem operasi kecil dengan struktur modular yang cukup kompleks.

Hasil search system/main.c

The screenshot displays the SourceTrail IDE interface. The main window shows the `main.c` file with the following code:

```
1 /* main.c - main */
2
3 #include <xinu.h>
4
5 process main(void)
6 {
7     /* Run the Xinu shell */
8
9     recvcr();
10    resume(create(shell, 8192, 50, "shell", 1, CONSOLE));
11
12    /* Wait for shell to exit and recreate it */
13
14    while (TRUE) {
15        receive();
16        sleepss(200);
17        kprintf("\n\nMain process recreating shell\n\n");
18        resume(create(shell, 4096, 20, "shell", 1, CONSOLE));
19    }
20    return OK;
21 }
22
23
```

Below the code editor, the 'References' panel shows one reference to `main.c` at line 3, indexed, with the translation unit `D:/Kuliah/Semester 6/Sistem Operasi/xinu/device/eth/ethcontrol.c`.

The 'Errors' panel at the bottom shows two fatal errors:

Type	Message	File	Line	Indexed	Translation Unit
FATAL	"xinu.h" file not found	D:/Kuliah/Semester 6/Sistem Operasi/xinu/device/eth/ethcontrol.c	3	yes	D:/Kuliah/Semester 6/Sistem Operasi/xinu/device/eth/ethcontrol.c
FATAL		D:/Kuliah/Semester 6/Sistem Operasi/xinu/			

The status bar at the bottom indicates 'displaying 202 errors (202 fatal)' and 'No IDE connected'.

Berdasarkan hasil eksplorasi pada file `main.c`, fungsi `main()` merupakan entry point pada sistem operasi Xinu. Fungsi ini bertugas untuk menjalankan shell dengan cara membuat proses menggunakan `create()` dan mengeksekusinya dengan `resume()`. Program berjalan dalam loop tak hingga untuk memastikan shell tetap aktif.

Function recvclr

The screenshot displays the SourceTrail IDE interface. The top menu bar includes Project, Edit, View, History, Bookmarks, and Help. The main editor window shows the `recvclr` function in `recvclr.c`. The function is defined as `umsg32 recvclr(void)` and contains logic to clear incoming messages and return them if one is waiting. It uses a semaphore to manage communication. The function is referenced by `main` and `rdsread` in `rdsread.c`. The bottom status bar shows 202 fatal errors, with a message indicating that the `"xmu.h"` file was not found.

```
1 /* recvclr.c - recvclr */
2
3 #include <xmu.h>
4
5 /*-----
6  * recvclr - Clear incoming message, and return message if one waiting
7  *-----
8  */
9 umsg32 recvclr(void)
10 {
11     intmask mask; /* Saved interrupt mask */
12     struct proctent *prptr; /* Ptr to process's table entry */
13     umsg32 msg; /* Message to return */
14
15     mask = disable();
16     prptr = Aprocstab[currpid];
17     if (prptr->prhasmsg == TRUE) {
18         msg = prptr->prmsg; /* Retrieve message */
19         prptr->prhasmsg = FALSE; /* Reset message flag */
20     } else {
21         msg = OK;
22     }
23     restore(mask);
24     return msg;
25 }
```

References:

- `main` (in `rdsread.c`)
- `rdsread` (in `rdsread.c`)

Type	Message	File	Line	Indexed	Translation Unit
FATAL	"xmu.h" file not found	D:/Kuliah/Semester 6/Sistem Operasi/xmu/device/ethcontrol.c	3	yes	D:/Kuliah/Semester 6/Sistem Operasi/xmu/device/ethcontrol.c

displaying 202 errors (202 fatal)

Fungsi `recvclr()` digunakan untuk mengambil pesan yang diterima oleh suatu proses dan sekaligus menghapus status pesan tersebut. Fungsi ini juga mengatur interrupt untuk menjaga konsistensi data selama proses berlangsung.

Function sleepms

The screenshot displays the SourceTrail IDE interface. The main editor window shows the implementation of the `sleepms` function in `sleep.c`. The function is designed to delay the calling process for a specified number of milliseconds. It uses `syscall` to invoke `fcntl` with `delay` in milliseconds, then enters a loop where it checks if the delay is less than 0 (returning `SYSEERR`), if it is 0 (returning `OK`), or if it is greater than 0 (yielding the process). The function also handles interrupt masks and process state changes using `proctab`.

```
.. sleep.c
22 /*-----
23 * sleepms - Delay the calling process n milliseconds
24 *-----
25 */
26 syscall sleepms
27 {
28     int32 delay; /* Time to delay in msec. */
29     intmask mask; /* Saved interrupt mask */
30
31     if (delay < 0) {
32         return SYSEERR;
33     }
34     if (delay == 0) {
35         yield();
36         return OK;
37     }
38     /* Delay calling process */
39
40     mask = disable();
41     if (insertd(curprid, sleepq, delay) == SYSEERR) {
42         restore(mask);
43         return SYSEERR;
44     }
45     proctab[curprid].prstate = PR_SLEEP;
46     resched();
47     restore(mask);
48     return OK;
49 }
```

On the left, a call graph shows a call to `sleepms` from an unknown source, which then calls `resched` and `yield`.

The bottom status bar shows two fatal errors:

Type	Message	File	Line	Indexed	Translation Unit
FATAL	"xinu.h" file not found	D:/Kuliah/Semester 6/Sistem Operasi/xinu/device/eth/ethcontrol.c	3	yes	D:/Kuliah/Semester 6/Sistem Operasi/xinu/device/eth/ethcontrol.c
FATAL		D:/Kuliah/Semester 6/Sistem Operasi/xinu/			

Fungsi `sleepms()` digunakan untuk menunda eksekusi suatu proses dalam satuan milidetik. Proses yang memanggil fungsi ini akan diubah statusnya menjadi sleep dan scheduler akan menjalankan proses lain menggunakan `resched()`.

Fungsi resched

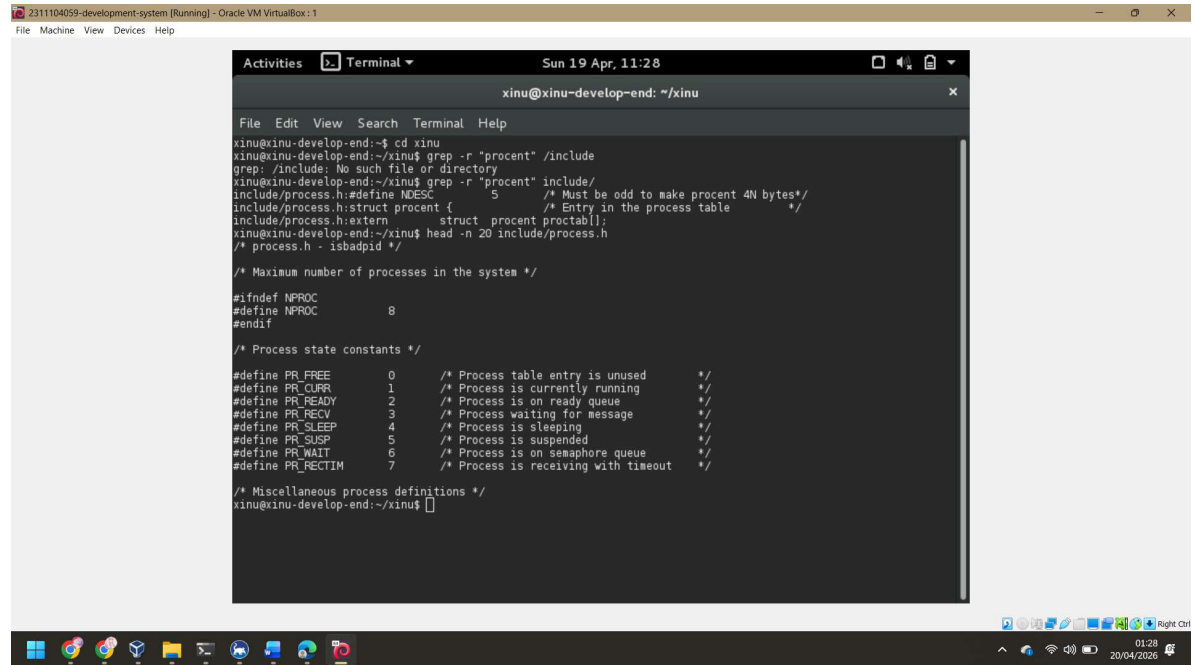
The screenshot displays the SourceTrail IDE interface for the 'resched' function. On the left, a call graph shows 'resched' as the central function, with arrows pointing to it from 'clkhandler', 'ready', 'receive', 'recvtime', 'sleepms', 'suspend', 'wait', and 'yield'. The main editor on the right shows the source code of 'resched' in 'resched.c', which is currently empty. Below the editor, a list of references shows that 'resched' is called from 'clkhandler.c', 'ready.c', 'receive.c', 'recvtime.c', 'sleep.c', 'suspend.c', 'wait.c', and 'wait.c'. The bottom status bar indicates 'displaying 202 errors (202 fatal)'.

Type	Message	File	Line	Indexed	Translation Unit
FATAL	"xinu.h" file not found	D:/Kuliah/Semester 6/Sistem Operasi/xinu/device/eth/ethcontrol.c	3	yes	D:/Kuliah/Semester 6/Sistem Operasi/xinu/device/eth/ethcontrol.c
FATAL		D:/Kuliah/Semester 6/Sistem Operasi/xinu/			

Fungsi `resched()` merupakan scheduler pada sistem operasi Xinu yang berfungsi untuk menentukan proses mana yang akan dijalankan selanjutnya. Fungsi ini dipanggil oleh berbagai fungsi lain seperti `sleepms()`, `yield()`, dan `receive()` untuk melakukan pergantian proses (context switching).

3. Carilah struktur data dari proses pada Xinu OS. Struktur data proses ada pada file apa? Informasi apa saja yang disimpan dalam struktur data tersebut? Hint: file berektensi .h Pada Linux bisa digunakan perintah “grep” Contoh: \$ grep -r “kata_yang_ingin_dicari” /path/ke/direktori/tempat/file/berada

Ini file yang mau di cari



The screenshot shows a terminal window titled "xinu@xinu-develop-end: ~/xinu" with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal output shows the following commands and results:

```
xinu@xinu-develop-end:~$ cd xinu
xinu@xinu-develop-end:~/xinu$ grep -r "procent" /include
grep: /include: No such file or directory
xinu@xinu-develop-end:~/xinu$ grep -r "procent" include/
include/process.h:#define NDESC      5      /* Must be odd to make procent 4N bytes*/
include/process.h:struct procent {      /* Entry in the process table */
xinu@xinu-develop-end:~/xinu$ head -n 20 include/process.h
/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC      8
#endif

/* Process state constants */

#define PR_FREE      0      /* Process table entry is unused */
#define PR_CURR      1      /* Process is currently running */
#define PR_READY      2      /* Process is on ready queue */
#define PR_RECV      3      /* Process waiting for message */
#define PR_SLEEP      4      /* Process is sleeping */
#define PR_SUSP      5      /* Process is suspended */
#define PR_WAIT      6      /* Process is on semaphore queue */
#define PR_RECVTIM      7      /* Process is receiving with timeout */

/* Miscellaneous process definitions */
xinu@xinu-develop-end:~/xinu$
```

The terminal window is part of an Oracle VM VirtualBox interface, with a taskbar at the bottom showing various application icons and a system clock indicating 01:28 on 20/04/2026.

cat include/process.h

```
xinu@xinu-develop-end:~/xinu$ cat include/process.h
/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC      8
#endif

/* Process state constants */

#define PR_FREE      0 /* Process table entry is unused */
#define PR_CURR      1 /* Process is currently running */
#define PR_READY      2 /* Process is on ready queue */
#define PR_RECV      3 /* Process waiting for message */
#define PR_SLEEP      4 /* Process is sleeping */
#define PR_SUSP      5 /* Process is suspended */
#define PR_WAIT      6 /* Process is on semaphore queue */
#define PR_RECTIM     7 /* Process is receiving with timeout */

/* Miscellaneous process definitions */

#define PNMLEN      16 /* Length of process "name" */
#define NULLPROC     0 /* ID of the null process */

/* Process initialization constants */

#define INITSTK      65536 /* Initial process stack size */
#define INITPRIO     20 /* Initial process priority */
#define INITRET      userret /* Address to which process returns */

/* Inline code to check process ID (assumes interrupts are disabled) */

#define isbadpid(x)  ((pid32)(x) < 0) || \
                    ((pid32)(x) >= NPROC) || \
                    (proctab[(x)].prstate == PR_FREE))

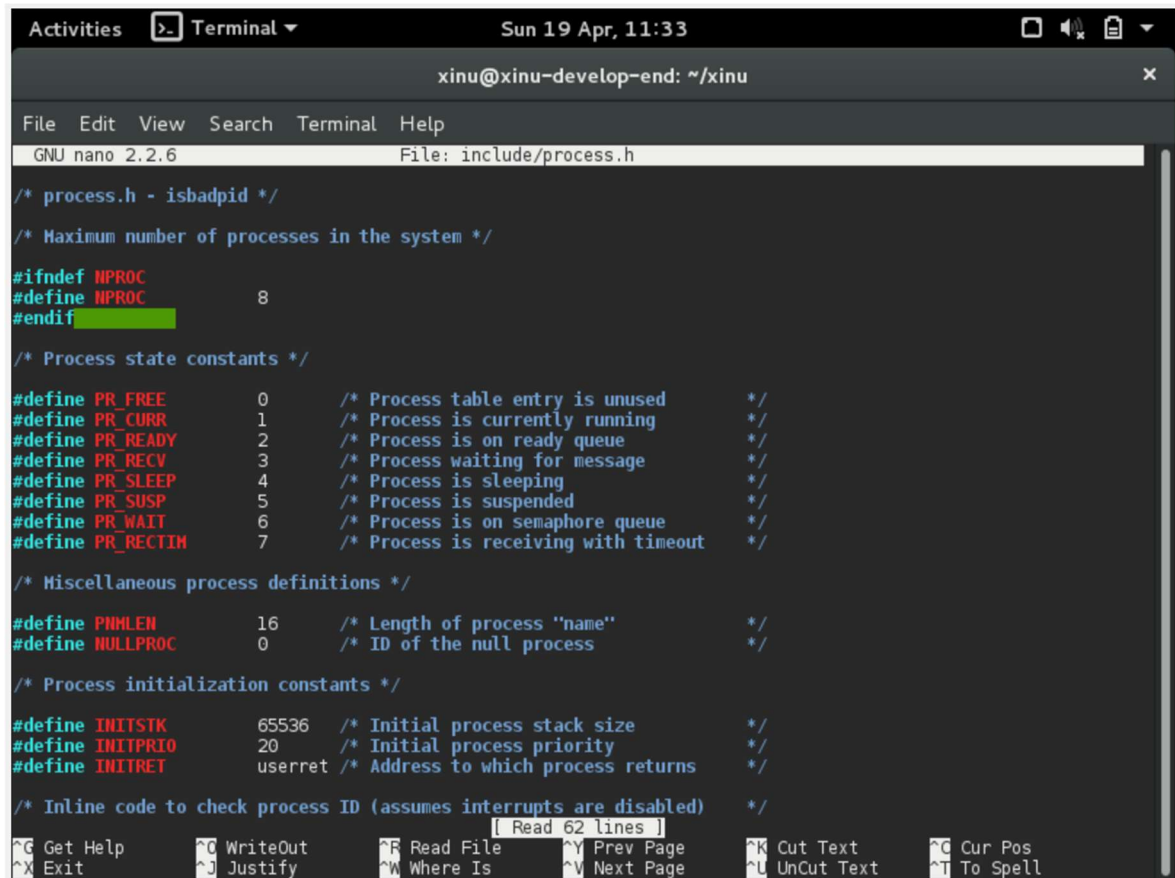
/* Number of device descriptors a process can have open */

#define NDESC        5 /* Must be odd to make proctent 4N bytes */

/* Definition of the process table (multiple of 32 bits) */

struct proctent {
    uint16 prstate; /* Entry in the process table */
    uint16 prprio; /* Process state: PR_CURR, etc. */
    char *prstkptr; /* Process priority */
    char *prstkbase; /* Saved stack pointer */
    uint32 prstklen; /* Base of run time stack */
    char pname[PNMLEN]; /* Stack length in bytes */
    /* Process name */
}
```

nano include/process.h



```
Activities Terminal Sun 19 Apr, 11:33
xinu@xinu-develop-end: ~/xinu

File Edit View Search Terminal Help
GNU nano 2.2.6 File: include/process.h

/* process.h - isbadpid */

/* Maximum number of processes in the system */
#ifndef NPROC
#define NPROC 8
#endif

/* Process state constants */
#define PR_FREE 0 /* Process table entry is unused */
#define PR_CURR 1 /* Process is currently running */
#define PR_READY 2 /* Process is on ready queue */
#define PR_RECV 3 /* Process waiting for message */
#define PR_SLEEP 4 /* Process is sleeping */
#define PR_SUSP 5 /* Process is suspended */
#define PR_WAIT 6 /* Process is on semaphore queue */
#define PR_RECTIM 7 /* Process is receiving with timeout */

/* Miscellaneous process definitions */
#define PNMLEN 16 /* Length of process "name" */
#define NULLPROC 0 /* ID of the null process */

/* Process initialization constants */
#define INITSTK 65536 /* Initial process stack size */
#define INITPRIO 20 /* Initial process priority */
#define INITRET userret /* Address to which process returns */

/* Inline code to check process ID (assumes interrupts are disabled) */

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^L UnCut Text ^T To Spell
```

Struktur data proses pada Xinu OS terdapat pada file process.h di direktori include. Hal ini ditemukan menggunakan perintah `grep -r "struct procent" ..` Struktur tersebut bernama struct procent dan berfungsi sebagai Process Control Block (PCB) yang menyimpan informasi seperti status proses, prioritas, stack, nama proses, parent ID, dan pesan.

4. Mengubah welcome banner pada Xinu
 - a. Carilah file yang menyimpan banner Xinu!
Hint: file berekstensi .h pada direktori xinu/include
 - b. Carilah file yang menampilkan banner Xinu!
Hint: file berekstensi .c pada direktori xinu/shell
 - c. Ubahlah welcome banner Xinu sehingga menjadi seperti ini:

```

Hello World!
Im the first Xinu app and running function main() in system/main.c.
I was created by nulluser() using create().
My creator will turn itself into the do-nothing null process.
I will create a second XINU app that runs shell() in shell/shell.c as an example.
You can do something else or do nothing; it's completely up to you.
...creating a shell

----- NAMA -----
X I N U
----- NIM -----

Welcome to Xinu!

```

Tambahkan NAMA dan NIM masing-masing, kemudian ubah banner Xinu sesuai dengan selera Anda

- d. Setelah melakukan perubahan lakukan perintah berikut ini pada terminal Linux :
 - \$ cd xinu/compile
 - \$ make clean
 - \$ make
 - \$ sudo minicom
 - Jalankan vm backend

Jawaban nomor 4:

- a. Cari file yang menyimpan banner

Filenya adalah shell.h

```

xinu@xinu-develop-end:~/xinu$ nano include/process.h
xinu@xinu-develop-end:~/xinu$ cd include
xinu@xinu-develop-end:~/xinu/include$ ls
arp.h      delay.h    file.h     io.h       multiboot.h  process.save  rdisksys.h  stdarg.h  tty.h
bufpool.h  device.h  flash.h    ip.h       name.h       prototypes.h  resched.h   stddef.h  uart.h
clock.h    dhcp.h    i386.h     kernel.h   net.h        pxe.h        rfilesys.h  stdio.h   udp.h
conf.h     dns.h     icmp.h     lfilesys.h pci.h        quark_irq.h  sdmc.h      stdlib.h  xinu.h
ctype.h    e1000e.h icu.h       limits.h   pipe.h       quark_pdat.h semaphore.h  string.h
date.h     ether.h   icu.s       mark.h     ports.h      queue.h      shell.h     testsuite.h
debug.h    ethloop.h interrupt.h memory.h    process.h    ramdisk.h    shprototypes.h  tftp.h
xinu@xinu-develop-end:~/xinu/include$

```

- b. Cari file yang menampilkan banner

```
xinu@xinu-develop-end: ~/xinu/include$ cd ../shell
xinu@xinu-develop-end: ~/xinu/shell$ ls
addargs.c      xsh_archo.c    xsh_date.c     xsh_help.c     xsh_memstat.c  xsh_ps.c       xsh_udpdump.c
lexan.c        xsh_arp.c      xsh_dumpevmp.c xsh_kill.c     xsh_netinfo.c  xsh_rdstest.c  xsh_udpcho.c
Notes On Pipes xsh_cat.c      xsh_echo.c     xsh_ls.c       xsh_ns.c       xsh_sleep.c    xsh_udpsrvr.c
shell.c         xsh_clear.c    xsh_exit.c     xsh_memdump.c  xsh_ping.c     xsh_tee.c      xsh_uptime.c
xinu@xinu-develop-end: ~/xinu/shell$
```

Ini shell.c yang di cari cari, file shell.c bakal nampilin banner ke terminal

- c. UBAH BANNER (INI BAGIAN UTAMA)

Saya pake nano ../include/shell.h

```

GNU nano 2.2.6                               File: ../include/shell.h
/* shell.h - Declarations and constants used by the Xinu shell */

/* Size constants */

#define SHELL_BUFLEN    TY_IBUFLEN+1    /* Length of input buffer      */
#define SHELL_MAXTOK    32              /* Maximum tokens per line    */
#define SHELL_CMDSTK    8192            /* Size of stack for process  */
/*      that executes command    */
#define SHELL_ARGLEN    (SHELL_BUFLEN+SHELL_MAXTOK) /* Argument area      */
#define SHELL_CHDPRIO    20             /* Process priority for command */

/* Message constants */

/* Shell banner (assumes VT100) */

#define SHELL_BAN0      "\033[31;1m"
#define SHELL_BAN1      ""
#define SHELL_BAN2      ""
#define SHELL_BAN3      ""
#define SHELL_BAN4      ""
#define SHELL_BAN5      ""
#define SHELL_BAN6      ""
#define SHELL_BAN7      ""
#define SHELL_BAN8      ""
#define SHELL_BAN9      "\033[0m\n"

/* Messages shell displays for user */

#define SHELL_PROMPT     "xsh $ "      /* Command prompt            */
#define SHELL_STRTHSG     "Welcome to Xinu!\n" /* Welcome message          */
#define SHELL_EXITMSG     "Shell closed\n" /* Shell exit message        */

[ Read 77 lines ]

```

Edit:

```
/* Shell banner (assumes VT100) */

#define SHELL_BAN0    "\033[31;1m"
#define SHELL_BAN1    "-----ZHAFIR ZAIDAN AVAIL-----"
#define SHELL_BAN2    " "
#define SHELL_BAN3    "  \  \  \  /  |  |  |  \  \  |  |  |  |  |  "
#define SHELL_BAN4    "  \  \  \  \  |  |  |  \  \  |  |  |  |  |  "
#define SHELL_BAN5    "  \  \  \  \  |  |  |  \  \  |  |  |  |  |  "
#define SHELL_BAN6    "  \  \  \  \  |  |  |  \  \  |  |  |  |  |  "
#define SHELL_BAN7    "  \  \  \  \  |  |  |  \  \  |  |  |  |  |  "
#define SHELL_BAN8    "-----2311104059-----"
#define SHELL_BAN9    "\033[0m\n"

/* Messages shell displays for user */

#define SHELL_PROMPT    "xsh $ "          /* Command prompt */
#define SHELL_STRTHSG    "Welcome to Xinu!\n" /* Welcome message */
#define SHELL_EXITMSG    "Shell closed\n" /* Shell exit message */

[ Read 77 lines ]

G Get Help      W WriteOut      R Read File      V Prev Page
X Exit          J Justify      W Where Is      V Next Page
C Cut Text      C Cur Pos
U UnCut Text    T To Spell
```

d. COMPILE & JALANKAN

```
Xinu for Vbox -- version #108 (xinu) Sun 19 Apr 14:58:36 EDT 2026

Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
  4447448 bytes of free memory. Free list:
    [0x0015E320 to 0x0009FFF7]
    [0x00100000 to 0x005FBFFF]
  115497 bytes of Xinu code.
    [0x00100000 to 0x0011C328]
  148488 bytes of data.
    [0x0011FEC0 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a00030f)

-----ZHAFIR ZAIDAN AVAIL-----

XINU

-----2311104059-----

Welcome to Xinu!

xsh $

|CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttySO
```

Anjay gacor